Towards a High Fidelity Training Environment for Autonomous Cyber Defense Agents

Sean Oesch
Oak Ridge National Laboratory
Oak Ridge, TN, United States
oeschts@ornl.gov

Amul Chaulagain
Oak Ridge National Laboratory
Oak Ridge, TN, United States
chaulagaina@ornl.gov

Phillipe Austria
Oak Ridge National Laboratory
Oak Ridge, TN, United States
austriaps@ornl.gov

Brian Weber
Oak Ridge National Laboratory
Oak Ridge, TN, United States
weberb@ornl.gov

Amir Sadovnik
Oak Ridge National Laboratory
Oak Ridge, TN, United States
sadovnika@ornl.gov

Cory Watson
Oak Ridge National Laboratory
Oak Ridge, TN, United States
watsoncl1@ornl.gov

Matthew Dixson
Oak Ridge National Laboratory
Oak Ridge, TN, United States
dixsonmk@ornl.gov

Benjamin Roberson
Oak Ridge National Laboratory
Oak Ridge, TN, United States
robersonbd@ornl.gov

ABSTRACT

Cyber defenders are overwhelmed by the frequency and scale of attacks against their networks. This problem will only be exacerbated as attackers leverage AI to automate their workflows. Autonomous cyber defense capabilities could aid defenders by automating operations and adapting dynamically to novel threats. However, existing training environments fall short in areas such as generalization, explainability, scalability, and transferability, making it intractable to train agents that will be effective in real networks. In this paper we take an important step towards creating autonomous cyber defense agents — we present a high fidelity training environment called *Cyberwheel* that includes both simulation and emulation capabilities. *Cyberwheel* simplifies customization of the training network and easily allows redefining the agent's reward function, observation space, and action space to support rapid experimentation of novel approaches to agent design. It also provides visibility into agent behaviors necessary for agent evaluation and sufficient documentation / examples to lower the barrier to entry. As an example use case of *Cyberwheel*, we present initial results training an autonomous agent to deploy cyber deception strategies in simulation.

ACM Reference Format:
Sean Oesch, Amul Chaulagain, Phillipe Austria, Brian Weber, Amir Sadovnik, Cory Watson, Matthew Dixson, and Benjamin Roberson. 2024. Towards a High Fidelity Training Environment for Autonomous Cyber Defense Agents. In *Workshop on Cyber Security Experimentation and Test (CSET 2024), August 13, 2024, Philadelphia, PA, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3675741.3675752

1 INTRODUCTION

As noted by prior work [13, 19], existing efforts towards autonomous cyber defense training environments fall short of providing the level of fidelity necessary to train agents that can generalize well or be efficiently transferred to an operational network. Shared training data and environments are critical in order to make advances in fields fueled by deep learning, as can be seen with ImageNet and computer vision in the early 2000s.[1] It is therefore imperative that researchers utilizing deep reinforcement learning to create autonomous cyber agents develop shared, high fidelity training environments. In this paper we present a high fidelity training environment called *Cyberwheel*[2] that aims to provide a shared resource to the autonomous cyber research community that is extensible, flexible, and adaptable.

Cyberwheel's simulation environment is built on robust network definition code and configuration files that enable rapid experimentation across topology sizes and configurations. The emulation environment uses a modified version of the Firewheel [4] test bed for emulation, which is an experimental platform built on QEMU that enables experimental reproducibility and the instantiation of networks with over 100K virtual nodes. The goal of our training environment is to aid the community in developing autonomous defense agents deployable in live networks that are (a) Adaptable to a variety of network sizes and topologies and (b) Adaptable to varying adversary strategies.

Section 2 provides an overview of shortcomings with existing training environments. Section 3 describes *Cyberwheel*'s attributes that address these shortcomings. And Section 4 provides a concrete use case in the *Cyberwheel* simulator in which the blue agent is trained to utilize cyber deception strategies. We plan to further build upon this preliminary work by integrating more blue agent actions and strategies, enabling reinforcement learning based red agents, and conducting full scale testing on our emulation environment. Pending organizational approval and the (soon to come) public release of Firewheel, we plan to open source our environment to enable community development and feedback.

CSET 2024, August 13, 2024, Philadelphia, PA, USA
2024. ACM ISBN 979-8-4007-0957-9/24/08
https://doi.org/10.1145/3675741.3675752

[1] https://www.historyofdatascience.com/imagenet-a-pioneering-vision-for-computers/
[2] https://github.com/ORNL/cyberwheel

2 BACKGROUND

While a significant amount of work has been done seeking to utilize reinforcement learning for autonomous cybersecurity [1, 3, 5–12, 14, 16, 20], a standardized open source environment that enables both simulation and emulation is still lacking. As we began our research into autonomous cyber defense using reinforcement learning, we experimented with several environments listed on the awesome RL for cyber repository [3]. Because we intended to use both simulation and emulation, we began with Farland [11], which we had access to through a working agreement. We then tested and extended the CybORG [18] environment, which has been used in several research papers and is discussed in more detail below because it is highlighted by Vyas et al. [19] as one of the better open source environments for training autonomous cyber agents, though it lacks emulation. We also examined the properties of several other available environments, but found them to be lacking in one or several of the following ways:

- Poor documentation combined with outdated imports and a lack of examples on how to extend the environment, often combined with dead code, makes it difficult to use.
- The way that the environment defines the experimental network, observation space, and agent actions is convoluted and difficult to extend cleanly.
- Functionality mentioned in the paper or ReadMe is not implemented. For example, CybORG mentions both emulation and a visualization tool, but neither is included in the repository.
- The environment is too focused on a specific question and cannot be easily extended to answer other research questions.
- The environment works well for environments with under 100 nodes, but is not easy to scale up to larger experiments and/or does not support running experiments in parallel.
- The environment lacks the granularity necessary to train agents that could be transferred to a real network. The environment may be too abstract or too granular. When too granular, the environment generally was written with a small set of experiments in mind and would take significant refactoring to modify.
- Not openly available to the research community.

While there are efforts underway to construct better training environments, such as the DARPA CASTLE program[4], their outcome is uncertain and their work not yet openly available. It therefore became necessary for us to develop a high fidelity simulation and emulation environment to support our research, which is the topic of this paper. The environment is called *Cyberwheel*. In the rest of this section, we provide an overview of our experience trying to extend CybORG and then provide a short primer on the reinforcement learning method we utilize in the example in Section 4.

2.1 CybORG

The Cyber Operations Research Gym (CybORG) [18] is a gym environment that aids in the creation of autonomous agents to help enhance cyber operations. CybORG provides a simulation environment to train autonomous agents for network defense. In our work, we initially tried using a modified version of CybORG provided in the second Cyber Autonomy Gym for Experimentation (CAGE) challenge [2] [17], which focused specifically on defending an enterprise network. We chose CybORG in part because, as noted by Vyas et al [19], it already contains a number of desirable properties, such as supporting new agent definitions, being open source with documentation, having the potential to support multi-agent reinforcement learning (MARL), and having a well-defined state. The CybORG environment used in CAGE Challenge 2, while a tremendous resource for the autonomous cyber defense community, has several shortcomings that resulted in us writing our own simulation environment:

(1) It does not enable the creation of diverse network topologies at scale.
(2) Inadequate red agent diversity or specificity.
(3) Lack of a functional emulation environment.
(4) Lack of visualization tools to observe agent behavior, which is critical for explainability to assess and diagnose potential issues with agent behavior.
(5) Observation space that scales poorly, is hard to translate to real networks & makes unrealistic assumptions about detection probabilities. Not trivial to modify the observation space to suit our needs.
(6) Dead code & lack of accurate documentation/examples. Not being maintained actively.

Several papers did utilize CybORG, such as the paper by Wolk et al. [21], but their modifications to CybORG were minor. They did not change the topology or make significant changes to the observation space or action space. We replicated their work, yet these minor changes to CybORG did not allow us to answer the types of research questions that we wanted to ask.

2.2 Reinforcement Learning

Reinforcement learning is a machine learning technique developed to address the challenge of learning from interaction with an environment in order to achieve long-term goals. For example, these long-term goals could include flying a helicopter safely to the intended destination, winning a game, efficiently managing a power station, or protecting a network against cyber attacks. The holy grail of RL is to create "generally" intelligent agents, where general intelligence may be defined as "an agent's ability to achieve goals in a wide range of environments". At a high level, RL agents map situations (states) to actions via a "policy" function in order to maximize a numerical reward signal. Rewards may vary in frequency (a single win/loss reward at the end of a game or a repeated "distance from goal" reward while a robot is walking) and complexity. There are many RL methods and in this paper we utilize the Proximal Policy Optimization (PPO) to demonstrate the usefulness of our environment.

Proximal Policy Optimization (PPO) is a reinforcement learning method that uses an actor-critic approach. The actor selects actions in the environment and the critic helps the actor select the actions that will result in the largest reward. In PPO, the actor first collects data by interacting with the environment. Then the critic estimates

[3] https://github.com/Limmen/awesome-rl-for-cybersecurity
[4] https://www.darpa.mil/program/cyber-agents-for-security-testing-and-learning-environments

the advantages of the various actions using this collected data. Using the critic's knowledge learned from the environment, the actor then creates a surrogate objective function that it optimizes in order to update its policy (the policy is what the actor uses to select actions). This process is then repeated to improve the overall reward achieved by the agent.

PPO is widely adopted because it has a high sample efficiency, tends to converge to a near optimal solution, works across a wide variety of problems/domains without hyperparameter tuning, and achieves a stable learning curve by limiting the size of a single policy update. PPO limits the size of a single policy update by using a clipped surrogate objective that limits the effect of a particular action becoming much more or less probable under a given policy. The clipped surrogate objective also allows multiple gradient ascent passes on a set of samples from the environment without causing overly large policy updates, which increases sample efficiency. For details on PPO, see the work of Schulman et al.[15].

3 *CYBERWHEEL* TRAINING ENVIRONMENT DESIGN

In this section we describe both the simulation environment (Section 3.1) and the emulation environment (Section 3.2) that comprise *Cyberwheel*.

3.1 Simulation Environment

3.1.1 Network Design. The primary goal of the network simulation portion of *Cyberwheel* is to strike a pragmatic balance between realism, ease of configuration, and performance. Motivating this goal is the desire to optimize training time by providing the agent with a large amount of realistic training experience in a reasonable amount of time. Adding too much detail to the simulation environment hurts episode runtime and makes configuration difficult, reducing training volume over time and ultimately hurts final agent performance. Including too little detail reduces the agent's adaptability to real-world network scenarios.

The *Cyberwheel* network is comprised of routers, subnets, and hosts, each of which is represented as a node on a networkx graph. Each of these components shares common attributes like a default route, routing table, and a firewall.

The main role of *Cyberwheel* routers is to route traffic between subnets. For each subnet that is connected to the router there is an associated router interface so that the router can communicate on said subnet. When simulated traffic attempts to traverse the router, the firewall state will be analyzed and if no matching rule is found the traffic will be dropped.

Subnets represent a single broadcast domain. They track which hosts are connected, which IPs are in use, and may also act as a DHCP server to avoid having to manually configure the IP of each host. Hosts can be configured in several different ways. In addition to the common attributes mentioned previously, each host also possesses the following: a parent subnet, list of defined services, and attributes that represent whether the host is a decoy and if it has been compromised by a red agent.

Hosts also have an optional host type attribute that can be thought of as a template that defines common host services and other host configurations. Using this attribute allows a large number of hosts to be easily configured in a predictable and reproducible manner. Further customization is still possible when leveraging the host type attribute. Finally, should additional granularity be needed for specific use-cases, each object described above is defined in classes that are easily extensible.

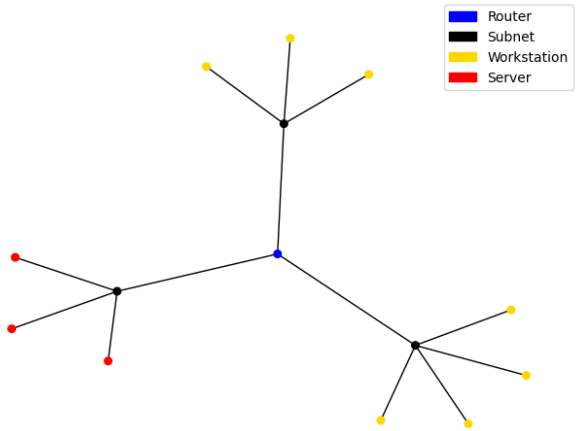

Figure 1: Graph of a network with 10 hosts, 3 subnets, and 1 router.

3.1.2 Scenario Configuration and Generation. In CybORG, networks are configured through YAML files that are parsed by the simulator and translated into Scenarios. These Scenario objects contain the starting parameters of the network simulation, including the subnets, the hosts, user accounts, processes, operating systems, and version information related to a given host. This level of granularity is useful for translation into an emulated environment. The YAML file also defines the action space available to each agent, the red agent policy characteristics, and the reward function to be used.

To allow for more ad-hoc network generation, we have created a script for *Cyberwheel* that is capable of generating a YAML file of a network. This script can define routers, subnets, hosts, interfaces between hosts, routes, and firewalls. A YAML file is automatically generated from all of these definitions and can be used by *Cyberwheel* immediately. This allows for sampling a diverse set of varying networks to train on. For example, the 10-host network displayed in Figure 1 was generated automatically using this script. Larger networks can also be generated. The largest network we created using it had 1 million total hosts and 2000 subnets.

As we continue to add functionality to the simulator, we require that any additional starting conditions, such as rewards for blue agents, be configurable in YAML files. By introducing this requirement, each set of configuration files acts as a detailed log of the environments that generate each blue agent policy. This will be invaluable for meta-analysis of blue agent policies. Also, by making generation characteristics configurable, we simplify future work on creating and evaluating training curriculum.

3.1.3 Alerts and Detectors. Existing environments require agents and observation spaces that observe the network directly. This creates major shortcomings and challenges when porting agents to real networks. First, using an agent trained in this way would require the installation and management of custom tools and sensors across every device on the network to provide the agent with the ability to act on and observe the network. In addition to the initial deployment, additions and changes to the agent and its action and obvservation space would require a complete redeployment of tools throughout the network, creating a vast amount of technical debt. Next, and perhaps more importantly, by requiring the agent to interpret raw network activity, the agent must act as a network intrusion detector in addition to providing action recommendations. This increases the "cognitive load" on the agent and prevents adopters from getting the most out of their existing investments in cybersecurity tools. Thus, our environment provides the ability to train agents based on observations from existing detectors by ingesting alerts, and provides an observation space which aligns neatly with popular classes of cyber-detection tools, such as host and network intrusion detection systems. To port to a real environment, the adopter needs only to provide a translation layer from their alert formats to the observation space of the agent.

Each red action is paired with what a perfect detection of that action would look like up to the limit of the simulation fidelity of our environment. For example, perfect alerts include what IPs, ports, and services are affected by the action along with the MITRE ATT&CK IDs associated with the action. Alerts are then passed to Detectors, which may deterministically or stochastically apply noise to the alerts, filter them out entirely, or include false positive alerts. For example, a "good" simulated NIDS detector may be defined which filters out host based activity from the agent, "detects" network-based activity with a high rate by passing them to the agent, and passes a small amount of false network-based alerts to the agent. Multiple detectors may be defined for a single environment. The modularity of our design allows action spaces, detectors, and observation spaces to easily be exchanged.

3.1.4 Blue Agent. In our environment, the blue agent needs to take an action from the RL agent, map that action to a blue action, execute that action, and return a tuple containing the action name, a unique identifier for the specific action, and whether or not the action was successful. The blue agent also defines a reward mapping that maps its actions to rewards. The method the blue agent uses to determine this mapping may vary from agent to agent depending on the requirements of the specific agent. This mapping is configured in a YAML file that is parsed when creating a new blue agent.

3.1.5 Rewards. Actions can have two types of rewards: immediate and recurring. Immediate rewards are gained on the same step that their action was executed on. Recurring rewards are instead calculated on the same step and all future steps until the end of the current episode or another action is taken that removes this reward. We incorporated both of these rewards to simulate both the upfront cost of perform an action and the lingering effects that action has. Deploying a decoy host, such as a honeypot, is an example of a blue action that has both types of rewards. The costs with starting the decoy would be represented by an immediate reward and maintaining it would be represented by a recurring reward. Red actions can also utilize both immediate and recurring rewards. For example, impacting a server could have a recurring reward.

A reward calculator needs to be defined in order to calculate each step's reward. This calculator requires mappings of red and blue actions to reward values. It then needs to define a method of actually calculating the reward for a step. In general, this requires the red and blue actions that were performed, whether the red and blue action was successful, and if the red action alerted the detector. Other operations, like keeping track of recurring rewards, may be defined by the reward calculator. The calculator must also define a reset method that specifies how to reset the reward calculator whenever the environment resets.

3.1.6 Red Actions. The red agent defines an adversary that is attacking a configured network. In our current stage of the simulation, it is deterministic and logic-based, with no RL training involved in its decision-making, though our code is written to make it straightforward to enable RL-based red agents. A red agent goes through a killchain of defined actions for each host to determine the actions it should take on a given host. Our current killchain implementation has the following four killchain phases: discovery, reconnaissance, privilege escalation, and impact.

The red actions are derived from Atomic Red Team (ART) [5] and MITRE ATT&CK [6] techiniques and tactics. There are currently 295 defined ART techniques, although many of them have not been fully written out into the logic of the game. Right now, the basic actions and killchain phases have been implemented into the red agent's logic. These basic actions are defined as Killchain Phases. These are higher-level, and can be attributed to multiple different ART Techniques, depending on various attributes of a given host such as the vulnerabilities associated, the services that are available to be exploited, or the operating system running.

ART techniques are defined with the following attributes:

- Mitre ID associated (e.g. "T1055.011")
- Technique Name (e.g. "Extra Window Memory Injection")
- MITRE Technique ID (e.g. "attack-pattern–0042a9f5-f053-4769-b3ef-9ad018dfa298")
- MITRE Data Components associated (e.g. ['OS API Execution'])
- Killchain Phase(s) used (e.g. ['defense-evasion', 'privilege-escalation'])
- MITRE Enterprise Mitigations
- Description
- Atomic Tests
- List of Common Vulnerabilities and Exposures (CVEs) [7] associated
- List of Common Weakness Enumerations (CWEs) [8] associated

Atomic tests define the data associated with a given technique, such as the supported platforms for an attack, required dependencies, as well as the actual shell commands that need to be run to

[5] https://atomicredteam.io/atomics/
[6] https://attack.mitre.org/
[7] https://cve.mitre.org/
[8] https://cwe.mitre.org/

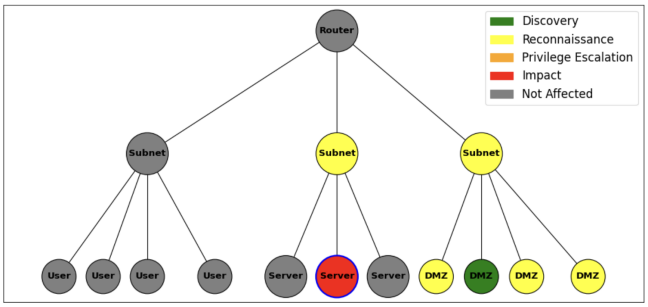

Figure 2: Visualization showing the red agent having impacted its targeted server. The blue outline indicates red agent position in the network.

execute the attack. These attributes are helpful for providing emulation necessities for the red agent to reflect the simulator in our emulator environment, such as implementing CWEs and CVEs in our emulated hosts and using the atomic test data to implement the red action as shell commands.

3.1.7 Visualization Tool. We created a visualization tool that enables autonomous agent creators to diagnose and understand agent behavior by replaying and observing episodes. This currently uses the Weights and Biases [9] API to get our trained model. This evaluation script allows various parameters to set for the environment, such as:

- Red Agent Type
- Network Configuration YAML
- Decoy Type Configuration YAML
- Host Info Configuration YAML
- Minimum number of Decoys
- Maximum number of Decoys
- Reward Scaling Value
- Reward Function

This script evaluates the model in an environment defined with the given parameters. The metadata and visualizations throughout the evaluation are saved locally in CSV and PNG files.

For visualizations, we made use of graphviz [10], an open-source graph visualization software, and pygraphviz, its python module, to neatly display our network with a hierarchical drawing layout.

The visualization portrays information of the network state across each step of an episode, allowing us to see the phase of the killchain each host is at in order to understand the level of intrusion throughout the simulation. It also allows the user to see which host the red agent is currently executing its attacks from.

3.1.8 Performance. We tested the performance of our simulator to determine per episode runtimes for different size networks. The episode runtime increases linearly with the number of hosts, as shown in Figure 3. We are currently running 128 environments in parallel during training, which could be increased with the appropriate hardware, with 100 steps per episode. The total training runtime

[9]https://wandb.ai/site
[10]https://graphviz.org/

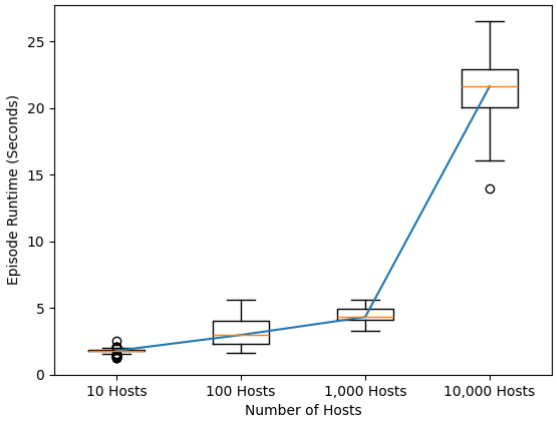

Figure 3: The per episode run time scales linearly with the size of the network (the X axis in the chart is log scale).

is dependent on the environment configuration for a given experiment (in addition to number of parallel environments) — agent definitions, steps per episode, reward function, hyperparameters and other factors.

3.2 Emulation Environment

Emulators provide a high-fidelity environment where agents trained in a simulator can be evaluated in real-world scenarios. Ideally, a trained agent would perform just as well as in an emulator as it did in a simulator, and adapt to new, unseen scenarios with little to no extra training. Currently, a limited number of emulators are being used in related research, including CybORG [18], Farland [11], and NASimEmu [8]. However, these emulators either fail to meet scalability requirements or are not open source. Furthermore, while it is possible to emulate scenarios by using a virtual machine (VM) manager such as Vagrant [11], it becomes increasingly difficult to manually configure and manage experiments as topologies grow in scale and complexity. For these reasons, we have chosen to use Firewheel [4].

Firewheel is a testbed designed and developed by Sandia National Laboratory (SNL). It provides infrastructure to emulate large-scale realistic network topologies and behaviors, and perform repeatable experiments. Firewheel comes with an extensive library of model components (e.g. hosts, routers and switches) to speed up the construction of common topology network components. Additionally, Firewheel provides tools to observe and collect data from experiments. The emulator uses Kernel-based Virtual Machine[12] (KVM) to virtualize host machines and Minimega[13], another SNL tool, to launch and manage VMs.

Our emulation architecture and environment, illustrated in Figure 4, was influenced by the work presented in [8]. The methodology involves the use of scenario configuration YAML files; the

[11]https://www.vagrantup.com/
[12]https://linux-kvm.org/
[13]https://www.sandia.gov/minimega/

same files used by the simulator portion of *Cyberwheel*. They define the network topology and services running on each host (e.g. SQL server, email, FTP, etc.). Before an experiment beings, the Scenario Converter parses the configuration files and generates a single *plugin* file, a special file which Firewheel uses to load and initialize VMs. Within the emulator, each VM is running a monitoring tool that captures events and generates logs. Futhermore, we have a designated VM for running a Security Information and Event Management (SIEM) solution. As of this workshop paper, we utilize Sysmon[14] for monitoring and the Elastic Stack[15] as a SIEM; however, these tools are subject to change as we further develop the emulation environment.

After the *plugin* file is generated and VMs are initialized, a scenario experiment can begin. Each step the Action Controller first receives an *action* from the Agent and maps it to a command that the attacker or defender can execute. In our implementation, commands are defined in python or shell scripts. Once executed, logs are generated, aggregated and processed by the SIEM. At the end of the step, the Observation Converter queries the SIEM and converts the query response into an *observation space* vector. The vector is consumed by the Agent in order to select a new *action*, at which point a new step begins.

We use Ansible[16] to deploy our Firewheel emulation environments. Ansible is an open-source configuration management automation tool. Much like Firewheel, Ansible has the capability to define a given configuration state declaratively. In addition, the Ansible modules used to define the emulation environments operate in an idempotent manner. This provides the capability to validate each environment and ensure that there are no unintended deviations from the desired state.

4 USE CASE - CYBER DECEPTION

Utilizing the *Cyberwheel* simulation environment, we evaluate a blue agent policy that deploys decoys in the network to delay or stop the adversary. The purpose of this section is not to make specific claims about the robustness of the trained agent, but to demonstrate how an experiment may be conceived and run inside of *Cyberwheel*. We describe in detail the design of the blue agent, red agent, and observation space, as well as an evaluation of different decoy types and the gamma hyperparameter. All experiments utilize the reinforcement learning algorithm PPO, described in Section 2.2.

4.1 Decoy Blue Agent and Reward

Our blue agent deploys decoys in order to trick the red agent into impacting a decoy server instead of an actual server. The blue agent is capable of deploying decoy hosts on the network. The types of hosts the blue agent can deploy is configurable by using a YAML to define the potential decoy host types. These decoys can be placed on any subnet and will appear to the red agent as a normal host.

The blue agent can perform three different actions. It can deploy a decoy, remove a decoy, or do nothing. Decoys can be deployed on any of the network's subnets. If the number of decoys deployed lies outside a specified range, then the reward calculator penalizes the blue agent by increasing the recurring reward of the decoys. This range's purpose is to have flexibility in how many decoys the blue agent can deploy, but to also prevent the blue agent from creating absurd numbers of decoys or from only choosing the nothing action. Decoys can be removed using the remove action. Because decoys of the same type on a subnet are not unique, this action simply removes one decoy of a specified type from the subnet. If there are no decoys of the specified type on the subnet, then this action fails decreasing the reward gained this step. If the agent chooses to do nothing, then the blue agent does not modify the network and no reward for taking this action is added.

The reward is calculated by summing the rewards for the blue agent performing its action, the red agent performing its action, and all of the current recurring actions. All of these rewards are negative. However, if a red action performs an action targeting a decoy, then that red action's reward is positive. This is because the goal in this scenario is to "trick" the red agent into interacting with a decoy rather than an actual server.

4.2 Red Agent

At the start of the game, the red agent has a foothold on a random user host in the network. This is represented in the form of a process on the host with *user* privileges. The actions available to the red agent are in the form of the phases of its killchain.

The *Discovery* killchain phase, given a host as the target, allows the red agent to either run a ping sweep on the target host's subnet, or a port scan on the target host. Running a ping sweep on a subnet will give the red agent a list of IP addresses for all of the hosts in the subnet, and add this data to the red agent's knowledge. Running a port scan on the target host will give the red agent information on the services currently running on the target host.

The *Reconnaissance* killchain phase, given a host as the target, allows the red agent to gather critical information about the target host. This currently includes a list of vulnerabilities, the host type, and any host interfaces associated. Having a list of vulnerabilities in the form of CVEs on a host allows for us to scale up the red agent actions with the ART techniques, and help determine whether an attack will be successful by checking that the host has a vulnerability that the attack exploits. Host interfaces, in this case, refer to a host having the IP address and connection capabilities routing to another host, regardless of whether it is on the same subnet. These interfaces are defined in the network configuration YAML file, and they enable the red agent to move between subnets if a corresponding interface is present.

Because the red agent's goal is to impact servers, it can use the host type gathered from *Reconnaissance* to help determine whether to move to a different host to attack it or not. When the red agent finds a server host, it will move to that host with a *Lateral Movement* action, which adds the malicious process to the server, making it the source of future attacks and actions. If it finds a user host, it will remain on the host and continue looking for a server.

When the red agent moves to a server, it will use *Privilege Escalation* to expand its privileges on the host. This involves elevating the malicious process's privilege level on the host from *user* to *root*. After this action, the red agent can use the *Impact* killchain phase on the target host, which is its overall goal throughout the game.

[14] https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon
[15] https://www.elastic.co/elastic-stack/
[16] https://www.ansible.com/

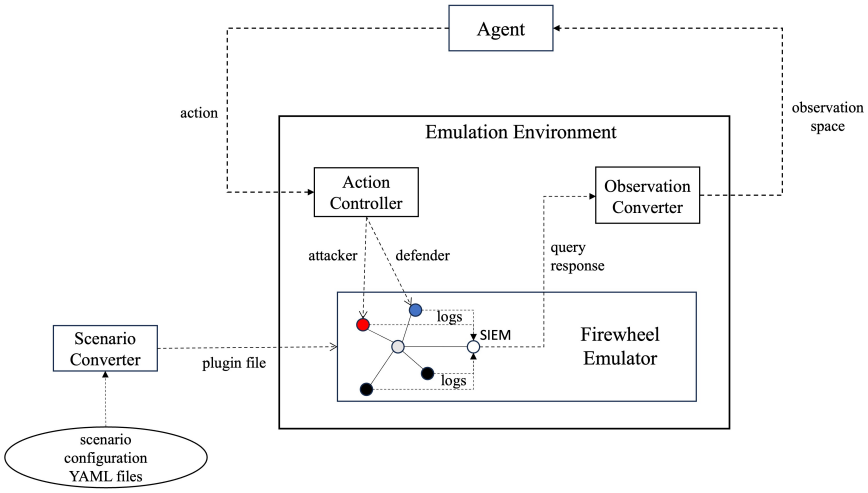

Figure 4: Emulation architecture using Firewheel.

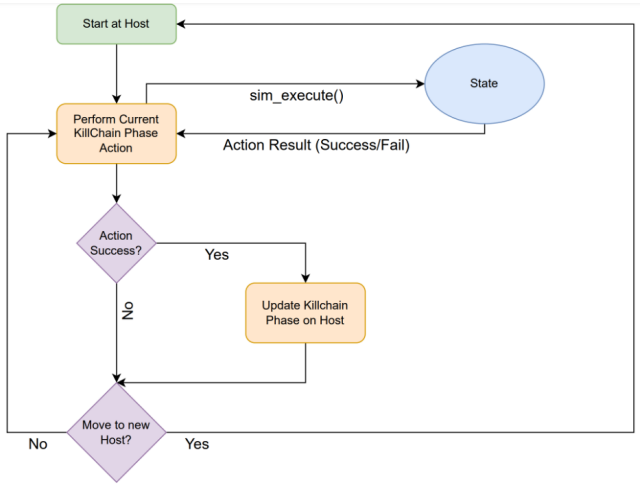

Figure 5: Killchain Agent Logic Flowchart

The default *KillChain* agent's goal is to target a server host. Once it finds and runs *Impact* on the server, it will continue to impact it for every action until the game is over. The *Recurring Impact* agent, however, is motivated by a goal of impacting all of the servers on a given network to create downtime for the network. Therefore, every *Impact* will add a recurring reward to the game every subsequent round. After impacting a server, the red agent will continue to explore the network in order to attack and impact other servers, while the effects of each *Impact* continue to accumulate in the background. This allows the red agent more opportunities to attack the network while also forcing the blue agent to adapt to a different strategy, maximize its rewards, and lessen the cost of the red agent's actions. Therefore, the blue agent is still motivated to continue defending the network even after the initial impact, to prevent further damage.

4.3 Detector

We designed two types of detectors to use with our decoy agent. The first is a detector that creates alerts if the target of the red action was a decoy. In other words, it detects when a host interacts with a decoy. The rationale behind this design is that non-compromised hosts are not meant to interact with decoys. If there is an interaction between a decoy and a host, it is likely that the source host is compromised.

The second type of detector is a probabilistic detector. Each red action has a set of techniques associated with it in order to simulate how the action would performed in the real-world. We used MITRE ATT&CK's tactics and techniques as a basis for assigning techniques to actions. This detector examines the techniques a red action uses and compares it against its own set of techniques. For each technique that the red action uses that is in the detector's set, there is a possibility that the detector creates an alert for that interaction. The techniques the detector can detect are defined in a YAML file that specifies the name of the technique and its probability of detection.

This configurability allows for probabilistic detectors to be created for a variety of scenarios. We created two config files to simulate a network and host based intrusion detection system.

4.4 Observation Space

Because our use case focuses on deception, the blue agent needs to place decoys in places where they are most likely to be accessed. This requires our observation space to include some history of where detectors have found the red agent to be. Our observation space's size is twice the number of non-decoy hosts on the network. The first half of the observation space represents the observations made at the current step. It consists of a single boolean for each non-decoy host on the network. If the detector produced an alert for that host this step, then its corresponding value in the observation space will be set to 1. Otherwise, it is 0. The second half of the observation space is reserved for history. Like the first half, the history portion of the observation space has a single boolean for each non-decoy host. However, it keeps track of if the detector

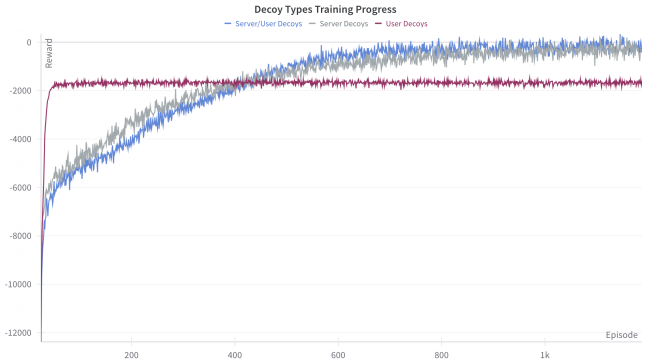

Figure 6: Training different decoy type deployments: only servers, only users, or either. The agent performs notably better at defending the network with server decoys compared to user decoys.

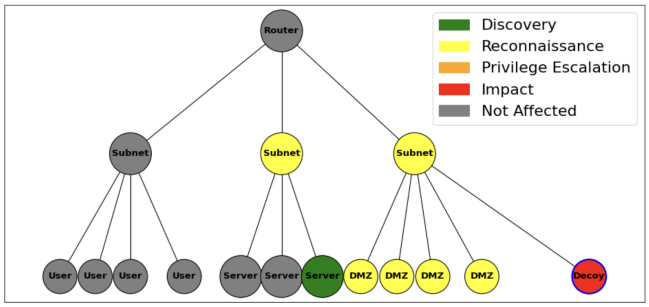

Figure 7: Server Decoy Deployment with Gamma = 0.9. The blue agent deploys a decoy on the DMZ subnet, redirecting the red agent's attention from the actual server targets.

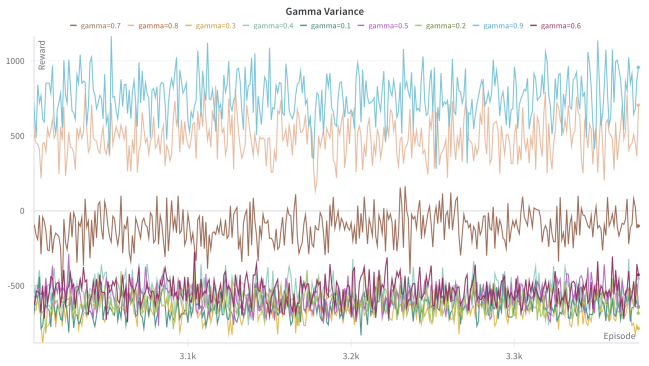

Figure 8: Training Results for varying gamma values. The higher gamma values perform much better than the lower gamma values, and results in decoys being deployed more in the DMZ subnet.

has ever produced an alert for a host for an entire episode. This information is intended to help the agent learn on which hosts red actions tend to occur and to place decoys in the red agent's path where they would be most effective.

4.5 Evaluations

The evaluations described below are all trained on a 13-host network with 3 subnets, one with primarily user hosts, one with server hosts, and one with DMZ hosts.

4.5.1 Evaluating Decoy Types. We tested the blue agent behavior when it could only deploy user decoy types, only deploy server decoy types, or able to deploy either decoy type. These tests were trained and evaluated with default parameters and a decoy range of 1-4 decoys.

When only deploying user decoys, it would deploy its decoys on the DMZ subnet to delay the red agent for a few steps and receive the marginal positive reward that it gave. However, the red agent would eventually move on to attack the server regardless of the blue agent's actions. When only deploying server decoy types, it would deploy its decoys on both the DMZ subnet and the user subnet. This way, it had a good chance of redirecting the red agent's path throughout its attack. Because the decoy was a server, the red agent would focus on impacting it instead of the actual servers in the network, which allowed it to be caught. Giving the blue agent the choice of deploying either a user or a server decoy did not change much for the agent behavior, as it learned to always choose to deploy a server decoy to actually stop the red agent.

4.5.2 Evaluating Gamma Variance. The gamma variable, also known as the discount factor, determines the emphasis an agent places on long-term rewards throughout training. A gamma value closer to 0 tends to prioritize short-term rewards, while a gamma value closer to 1 tends to prioritize long-term rewards. We trained and evaluated our RL agent with gamma values ranging from 0.1-0.9 to determine its effect on the behavior and reward.

For gamma values between 0.1 and 0.6, the trained blue agents all reached the same result: they did not learn any useful techniques to trick the red agent into impacting a decoy. With a gamma value of 0.7, the trained agent learned to deploy a server decoy on the user subnet to redirect the red agent's attention. With a gamma value 0.8, it learned to deploy these server decoys on either the DMZ or the user subnet, but mostly deployed them on the user subnet rather than the DMZ subnet. Around this stage, it can become probabilistic in determining whether the red agent finds the decoy server or a real server first, assuming that it has access to both. For a gamma value of 0.9, it displayed similar behavior, but mostly deployed the decoys on the DMZ subnet rather than the user subnet, and this method gave it the highest reward out of all of the gamma values tested.

5 FUTURE WORK

In this preliminary work, we summarized the shortcomings of existing training environments for autonomous cyber defense agents, presented the design for the *Cyberwheel* simulation and emulation environment, and shared results using the *Cyberwheel* simulation environment to train a blue agent to intelligently deploy decoys. Moving forward, we plan to work collaboratively with the autonomous cyber research community to extend *Cyberwheel* to

support additional blue and red agent actions and goals, reinforcement learning based red agents, and the transfer of agents from simulation to emulation to real networks with minimal retraining. To support this broader goal, we will continue to enhance the scalability and usabiltiy of *Cyberwheel* as we conduct our own research and receive feedback from others users. Specific research areas of interest to us are utilizing multiagent approaches and curriculum learning when training agents in larger and/or more complex network environments.

REFERENCES

[1] Elizabeth Bates, Vasilios Mavroudis, and Chris Hicks. 2023. Reward Shaping for Happier Autonomous Cyber Security Agents. *arXiv preprint arXiv:2310.13565* (2023).
[2] CAGE (Ed.). 2022. *TTCP CAGE Challenge 2.*
[3] Ashutosh Dutta, Samrat Chatterjee, Arnab Bhattacharya, and Mahantesh Halappanavar. 2023. Deep Reinforcement Learning for Cyber System Defense under Dynamic Adversarial Uncertainties. *arXiv preprint arXiv:2302.01595* (2023).
[4] Kasimir Georg Gabert, Adam Vail, Tan Q. Thai, Ian Burns, Michael J. McDonald, Steven Elliott, John Vivian Montoya, Jenna Marie Kallaher, and Stephen T. Jones. 2015. Firewheel - A Platform for Cyber Analysis. (11 2015). https://www.osti.gov/biblio/1333803
[5] Kim Hammar and Rolf Stadler. 2020. Finding effective security strategies through reinforcement learning and self-play. In *2020 16th International Conference on Network and Service Management (CNSM)*. IEEE, 1–9.
[6] Kim Hammar and Rolf Stadler. 2022. Learning Security Strategies through Game Play and Optimal Stopping. *arXiv preprint arXiv:2205.14694* (2022).
[7] Zhenguo Hu, Razvan Beuran, and Yasuo Tan. 2020. Automated penetration testing using deep reinforcement learning. In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2–10.
[8] Jaromír Janisch, Tomáš Pevnỳ, and Viliam Lisỳ. 2023. NASimEmu: Network Attack Simulator & Emulator for Training Agents Generalizing to Novel Scenarios. *arXiv preprint arXiv:2305.17246* (2023).
[9] Michael Kouremetis, Dean Lawrence, Ron Alford, Zoe Cheuvront, David Davila, Benjamin Geyer, Trevor Haigh, Ethan Michalak, Rachel Murphy, and Gianpaolo Russo. 2024. Mirage: cyber deception against autonomous cyber attacks in emulation and simulation. *Annals of Telecommunications* (2024), 1–15.
[10] Microsoft Defender Research Team. 2021. CyberBattleSim. *Created by Christian Seifert, Michael Betser, William Blum, James Bono, Kate Farris, Emily Goren, Justin Grana, Kristian Holsheimer, Brandon Marken, Joshua Neil, Nicole Nichols, Jugal Parikh, Haoran Wei* (2021).
[11] Andres Molina-Markham, Cory Miniter, Becky Powell, and Ahmad Ridley. 2021. Network environment design for autonomous cyberdefense. *arXiv preprint arXiv:2103.07583* (2021).
[12] Jakob Nyberg and Pontus Johnson. 2023. Training Automated Defense Strategies Using Graph-based Cyber Attack Simulations. *arXiv preprint arXiv:2304.11084* (2023).
[13] Sean Oesch, Phillipe Austria, Amul Chaulagain, Brian Weber, Cory Watson, Matthew Dixson, and Amir Sadovnik. 2024. The Path To Autonomous Cyber Defense. *arXiv preprint arXiv:2404.10788* (2024).
[14] Ahmad Ridley. 2018. Machine learning for autonomous cyber defense. *The Next Wave* 22, 1 (2018), 7–14.
[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
[16] Jonathon Schwartz and Hanna Kurniawati. 2019. Autonomous penetration testing using reinforcement learning. *arXiv preprint arXiv:1905.05965* (2019).
[17] Maxwell Standen. 2022. Cyber Autonomy Gym for Experimentation Challenge 2. https://github.com/cage-challenge/cage-challenge-2. Created by Maxwell Standen, David Bowman, Son Hoang, Toby Richer, Martin Lucas, Richard Van Tassel, Phillip Vu, Mitchell Kiely.
[18] Maxwell Standen, Martin Lucas, David Bowman, Toby J Richer, Junae Kim, and Damian Marriott. 2021. Cyborg: A gym for the development of autonomous cyber agents. *arXiv preprint arXiv:2108.09118* (2021).
[19] Sanyam Vyas, John Hannay, Andrew Bolton, and Professor Pete Burnap. 2023. Automated Cyber Defence: A Review. *arXiv preprint arXiv:2303.04926* (2023).
[20] Erich Walter, Kimberly Ferguson-Walter, and Ahmad Ridley. 2021. Incorporating Deception into CyberBattleSim for Autonomous Defense. *arXiv preprint arXiv:2108.13980* (2021).
[21] Melody Wolk, Andy Applebaum, Camron Denver, Patrick Dwyer, Marina Moskowitz, Harold Nguyen, Nicole Nichols, Nicole Park, Paul Rachwalski, Frank Rau, et al. 2022. Beyond cage: Investigating generalization of learned autonomous network defense policies. *arXiv preprint arXiv:2211.15557* (2022).